University of Southern Denmark

Department of Mathematics and Computer Science

DM546: Compiler Construction

Compiler Construction - Final Report

Authors:

Nel Skowronek
nesko25@student.sdu.dk

Jan Ryszkiewicz
jarys25@student.sdu.dk

December 7, 2025

1 Preliminaries

We implemented the interpreter using standard tools and libraries available in Java. No parser generators or external frameworks were used.

2 Technical Description

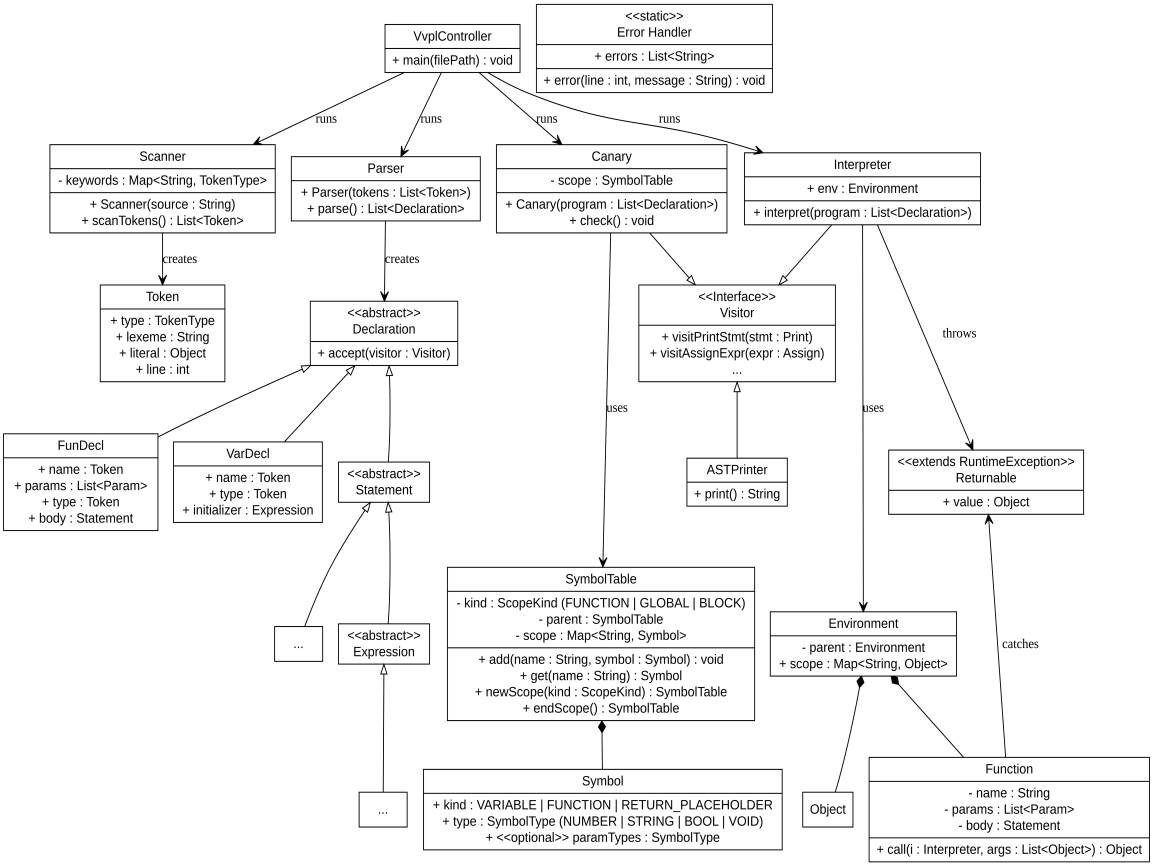
2.1 Architecture

As shown below on the diagrams - the main class `VVPLController` takes in the filepath of the file to interpret and:

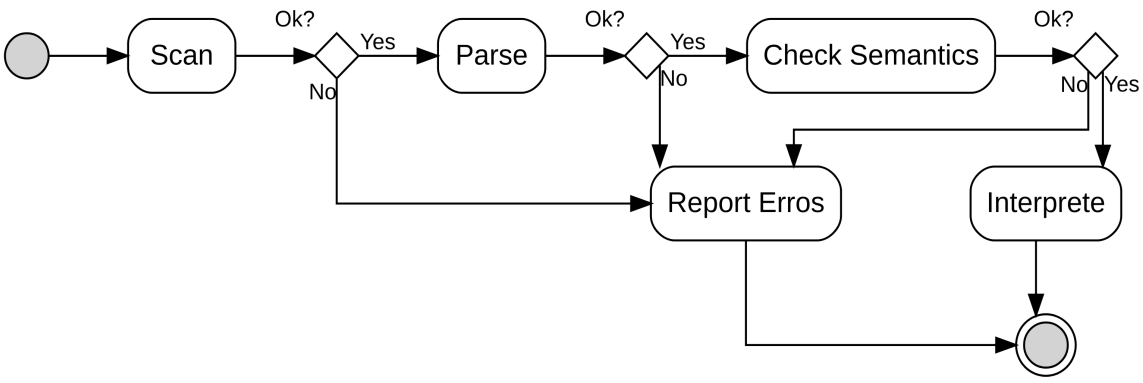
1. Runs the source code through the `Scanner` that generates tokens
2. Only when the `Scanner` succeeded - runs the tokens through the `Parser` that generates the Abstract Syntax Tree (it's basic parent node being `Declaration`)
3. Only when the `Parser` succeeded - runs the `Declarations` (AST) through the Semantic Checker (called `Canary` here, because it checks everything at once before the final evaluation - like the bird)
4. And finally, only when `Canary` succeeded - runs the `Interpreter` on the AST

If at any point in this chain of execution an error appears - the static `ErrorHandler` class is used to store all the errors and print them all at the end of a phase (eg. during parsing) to allow the program to check for all the error in that phase, not just the first encountered one. This is also visible on the activity diagram below.

2.1.1 Class Diagram



2.1.2 Activity Diagram



This clear division of phases allows the code to be Single-Resposibility and Open-Close as it makes it very easy to change how one phase is executed, or to insert an additional phase at any point - without influencing other classes.

Additionally the Visitor Pattern was used to visit the AST nodes, which keeps the implementation of the nodes clean and focused only on their data - allowing for easy implementation of the Semantic Checker, the Tree-Walk `Interpreter`, the `ASTPrinter` (for debugging purposes) and possible additional checkers if need be.

2.2 Implementation

2.2.1 Scanning

The `Scanner` goes through the source code character-by-character and handles each expected character in it's respective way via a simple switch-case. It matches single-character tokens right away, and for single or 2-character tokens (eg. '<' could be either '<' or '<=') it looks one character ahead. For longer and more complicated tokens (such as ids, keywords, literal strings and numbers) it reads all expected characters until a specified end (usually a whitespace or a quotation mark) and checks via a hash-map if the resulting lexeme is a keyword or not.

All whitespaces (except in strings) are ignored, and all unexpected characters result in a `ScanError`.

2.2.2 Parsing

We implemented a recursive-descent `Parser` with one lookahead. It implements the production rules as specified in the grammar by calling an appropriate recursive method for each rule (eg. `declaration()`, `ifStmt()`, `or()`...). All ambiguities are resolved by looking one token ahead and matching it with a production rule. All unexpected tokens that don't match any of the production rules result in a `ParseError`. At this point the `Parser` skips to the next semicolon token to "synchronize" to the next declaration.

For the `cast` expression a specific class for the AST was defined that has two children: `type` (leaf) and `value` (deriving from `expression`). This allows for very easy type-checking and interpreting of cast expressions as well as nested `cast` expressions.

The "dangling else" is assigned to **the most nested if**, and all following "elses" are assigned to the second most nested, the third, and so on.

2.2.3 Semantic Analysis

We perform the Scope Analysis and Type Analysis simultaneously, due to their similar nature and to avoid overhead of running the checker twice. But first we quickly go over the list of declarations and add the signatures of function declarations as `Symbol`s to the `SymbolTable` to allow for out-of-order declarations of functions in the global scope. Then we properly walk the tree and for every AST node we check the applicable properties:

- if a variable has been defined (we don't allow shadowing)
- if a function has been defined
- if a function is not being defined anywhere other than the global scope

- if function parameters have unique names
- if there is no unreachable code after a `return` statement
- if there is no return statement outside a function

And for each AST node, if none of the above applicable properties were violated, we check:

- if the initializer type matches the declared variable type
- if there is an initializer at all
- if binary operands are of correct type
- if logical operands are of correct type
- if a value can be casted
- if the type of returned value matches the declared function type

To do that, the Semantic Check `Visitor` returns a `SymbolType` when visiting any node, to pass information to the parent node about it's children's types. All declarations are stored in the `SymbolTable` and a new `SymbolTable` is created for each block or function and then destroyed at the end of the scope, disallowing the usage of variables defined in a deeper scope.

To handle the return statements, at the beginning of `visitFunDecl()` method a special symbol is added to the function's `SymbolTable` that acts as a placeholder for the return value and stores information about it's expected type. While visiting the `returnStmt` the checker can then check if the current scope is a function scope (it keeps that information) and extract the special symbol's type to compare it with the statement's `value` child node's type.

If a function's type is unspecified it is treated as `void`-type and all return statements in that function are expected to not have a value.

If any properties are violated the checker (similarly to `Scanner` and `Parser`) reports either a `ScopeError` or a `TypeError` to the `ErrorHandler`, and returns `null` as the type of the node. That way parent nodes know that something went wrong and can keep on looking for different violations, instead of propagating the same error higher.

2.2.4 Interpretation

As mentioned before, the `Interpreter` is an AST `Visitor` and executes the code by walking the tree. It assumes that everything that ought to have been checked during previous phases has been checked correctly and that no error appeared along the way.

That being said, the `Interpreter` is still checking for a few things:

- division by 0
- runtime evaluable casting rules (what boolean value to cast a number to, and if a string can be casted to a number)

And throws a `RuntimeError` and stops the execution if need be.

An `Environment` is used to map variable names to objects with their value at any given moment. Additionally, to handle functions, a new `Function` class has been declared (also mapped in the environment to the function name), that stores the function's declaration's AST node and interprets it (using the same interpreter, but a new, local environment) everytime it is called. The local environment initially contains only the mapping of parameter names (here we're allowing shadowing global variables) onto their respective values, given by the `callExpr`, and other globally declared functions.

Function class itself, when called, binds all arguments, copies current interpreter environment, flags etc. to be later retreived and creates a fresh environment for the interpreter with needed function declarations and variables that then runs the function in `inFunction = true` mode expecting a return value (if void, returns null). After function is evaluated it returns the interpreter to the state before function call, and sets the `inFunction` flag to false. This method allows reusage of existing instance of the interpreter and avoids recursive creation of new ones when calling a big recursive function (reduces stack usage).

The returns themselves are handled by `visitReturnStmt()` method that throws a special kind of error - `Returnable` - that remembers the value object that is being returned. That `Returnable` error is then caught by the mentioned above `Function` object. The returned value is extracted and returned back to the initial `callExpr`.

2.2.5 Monitoring and Logging

Due to the clear phase division, it is easy to add logging (printing the tokens or the AST) at any point of the interpretation process. However, we don't do that during the main program execution - the only things that appear is the output of the program, or the errors. The logging is used only for certain tests.

2.3 Testing & Evaluation

We created convenient test classes that dynamically create unit tests for all files in their respective `resources` directory. They run the tested component on the given `.in` file and compare the results with the given `.<component>` or `.out` file. We tried to test all major components, all AST nodes and almost all errors that are possible to appear.

The only thing we changed in the given tests was the way the errors were printed and their content - to align with our errors which we tried to make more descriptive. We made sure that that the error types and lines were identical.

3 Teamwork Organization

As our team consists of 2 people and there are 4 major components/phases of our interpretation process - each person worked on 2 of the components.

- **Nel**: Parsing and Semantic Checks
- **Jan**: Scanning and Interpreting

This division allowed to avoid potential merge-conflicts. Our workflow was organized through Git and we communicated regularly to ensure consistency in error handling and the interpretation pipeline itself.

While we were already familiar with regexes, we definitely learned a lot about finite automatas, grammars, production rules, IRs - but mostly about LL(1) grammars of course, since that was the grammar of the language we were interpreting.

4 Discussion

Alternatively to writing the scanner and the parser ourselves, we could've used various scanner and parser generators (like Flex and Bison for C) which could guarantee correctness, save time and work and teach us how to use a new framework. On the other hand, the downside of this option is that possibly, we would have to spend more time learning to use this technology and later it might have prove inefficient and a slight overkill for such a small language.

At this point adding anything new to the language and grammar would require substantial changes in each and every part of our program. Automation of Scanners and Parsers and more unified way of handling AST could be helpful if we ever decided to expand the VVPL.

Using Tree-Walk interpreter is also considered very inefficient, it'd be better to first translate the code into more LLVM-like structure and then run the ByteCode itself as it has a lot of built in optimizations we wouldn't have to worry about. Using try-catch methods as a way of returing functions also introduces a lot of overhead.

Overall, changing the general form of the interpreter and automating the Scanner and Parser could allow us to create a bigger, faster and easier to modify language (compiled into LLVM, then either interpreted or compiled to Machine Code using **clang**). Possible new features would include allowing user inputs or explicit memory allocation.